

**DESIGN AND DEVELOP AN AUTOMATED COLOR-BASED
PRODUCT SORTING SYSTEM FOR MANUFACTURING
INDUSTRIES. THE SYSTEM AUTOMATICALLY DETECTS AND
SORTS PRODUCTS BASED ON COLOR FOR INDUSTRIAL
AUTOMATION APPLICATIONS**

A PROJECT REPORT

Submitted by

MATHESH NISHANTH B – 24MER030

MOHAMMED SABEER S – 24MER031

PRAAVIN RAJAA G – 24MER039

PRASANNA T – 24MER042

DEVA PRIYAN R – 24MEL064

SATHISH KUMAR K – 24MEL067

DEPARTMENT OF MECHANICAL ENGINEERING



(Autonomous)

PERUNDURAI ERODE – 638 060

1. Project Overview

The Automated Color-Based Product Sorting System is an industrial automation system designed to automatically detect and sort products according to their color. The system uses a color sensor to identify the color of each product while it moves on a conveyor belt. A microcontroller processes the sensor information and controls actuators such as servo motors or pneumatic mechanisms to direct products into their respective sorting bins.

The system reduces manual sorting effort, improves sorting speed and accuracy, and minimizes human errors. It can be used in manufacturing, packaging, food processing, recycling, and material-handling industries. The project demonstrates the practical application of sensors, microcontrollers, conveyor systems, and automatic control in smart manufacturing.

2. Problem Statement

In many manufacturing industries, products are still sorted manually based on their color. Manual sorting is time-consuming, labor-intensive, and prone to human errors, especially when handling large quantities of products. It can also lead to inconsistent sorting accuracy and reduced production efficiency.

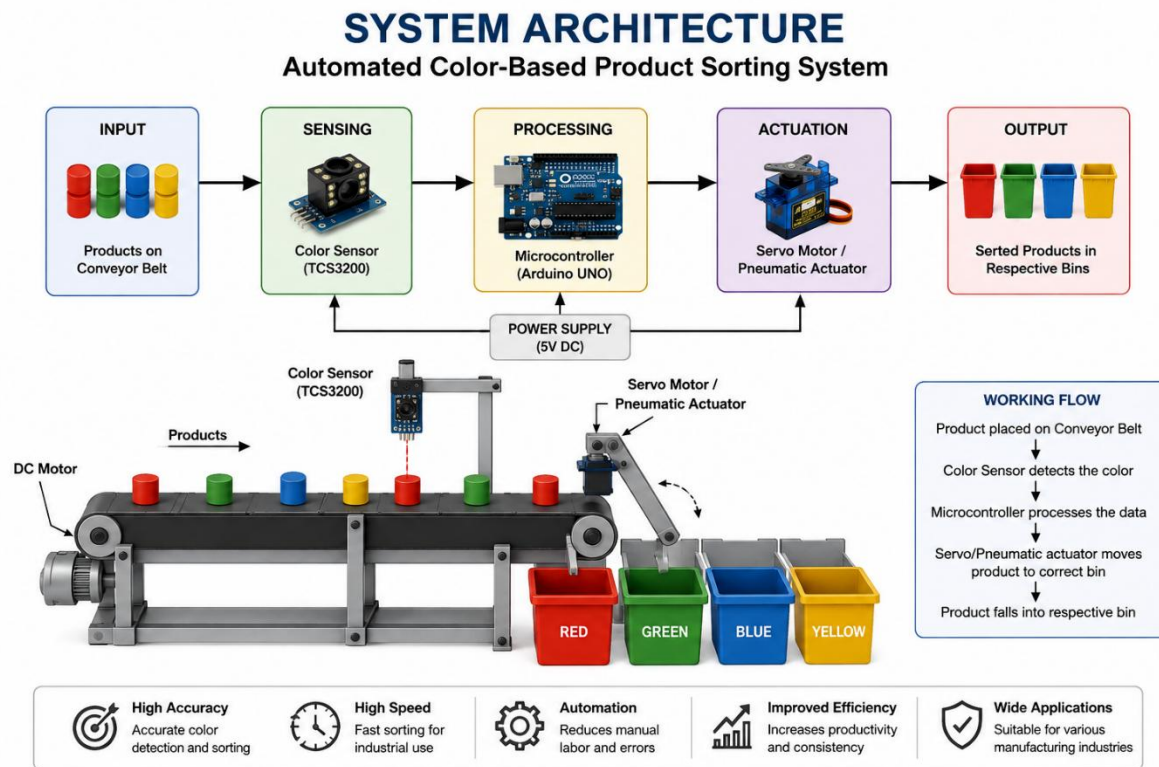
Therefore, there is a need for an automated color-based product sorting system that can accurately detect the color of products and automatically direct them into their respective bins. The proposed system aims to reduce human effort, improve sorting accuracy, increase production speed, and support efficient industrial automation.

3. Proposed Solution

The proposed system uses a color sensor, conveyor belt, microcontroller, and sorting mechanism to automatically identify and separate products based on their color.

- A conveyor belt moves the products through the sorting area.
- A color sensor detects the color of each product.
- The microcontroller processes the sensor signal and determines the product category.
- A servo motor or pneumatic actuator moves the product toward the correct sorting bin.
- Different bins are provided for different colors, such as Red, Green, Blue, and Yellow.
- The system continuously repeats the process for incoming products.
- This automation reduces manual labor and sorting errors while improving speed, accuracy, and productivity in manufacturing industries.

4. System Architecture

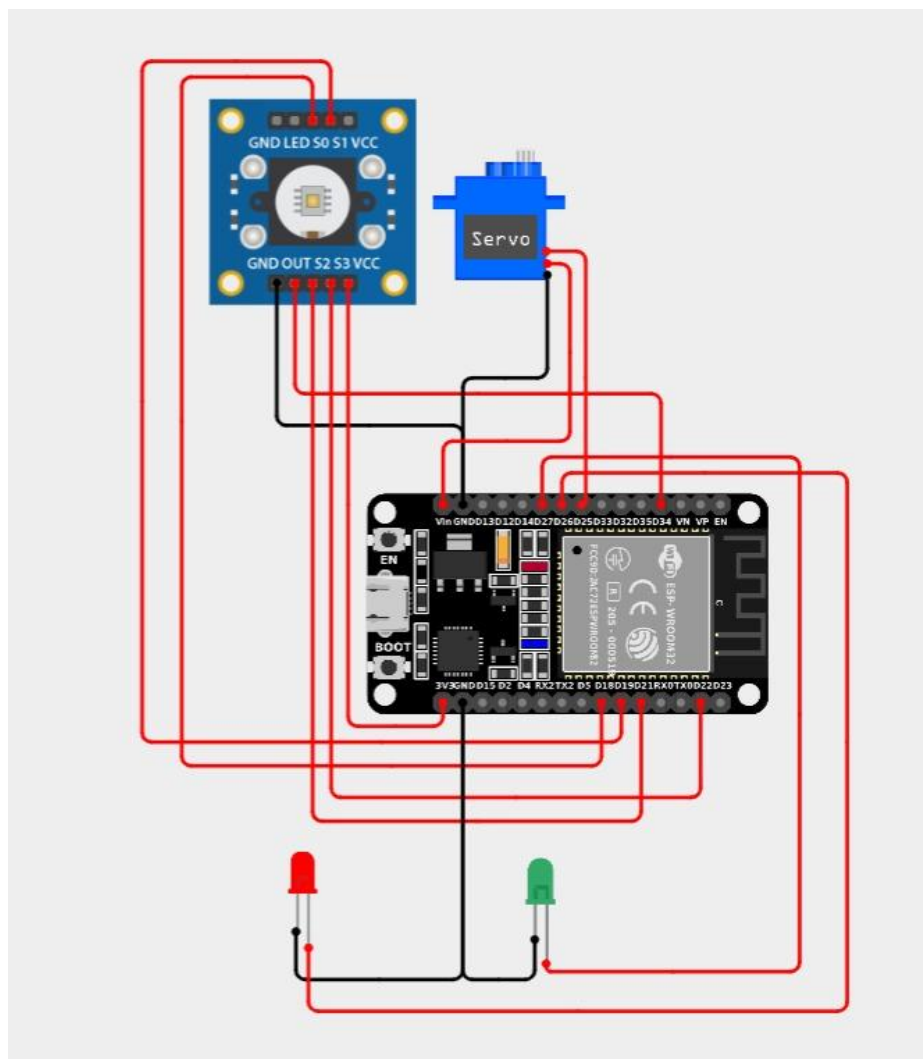


5. Circuit Table

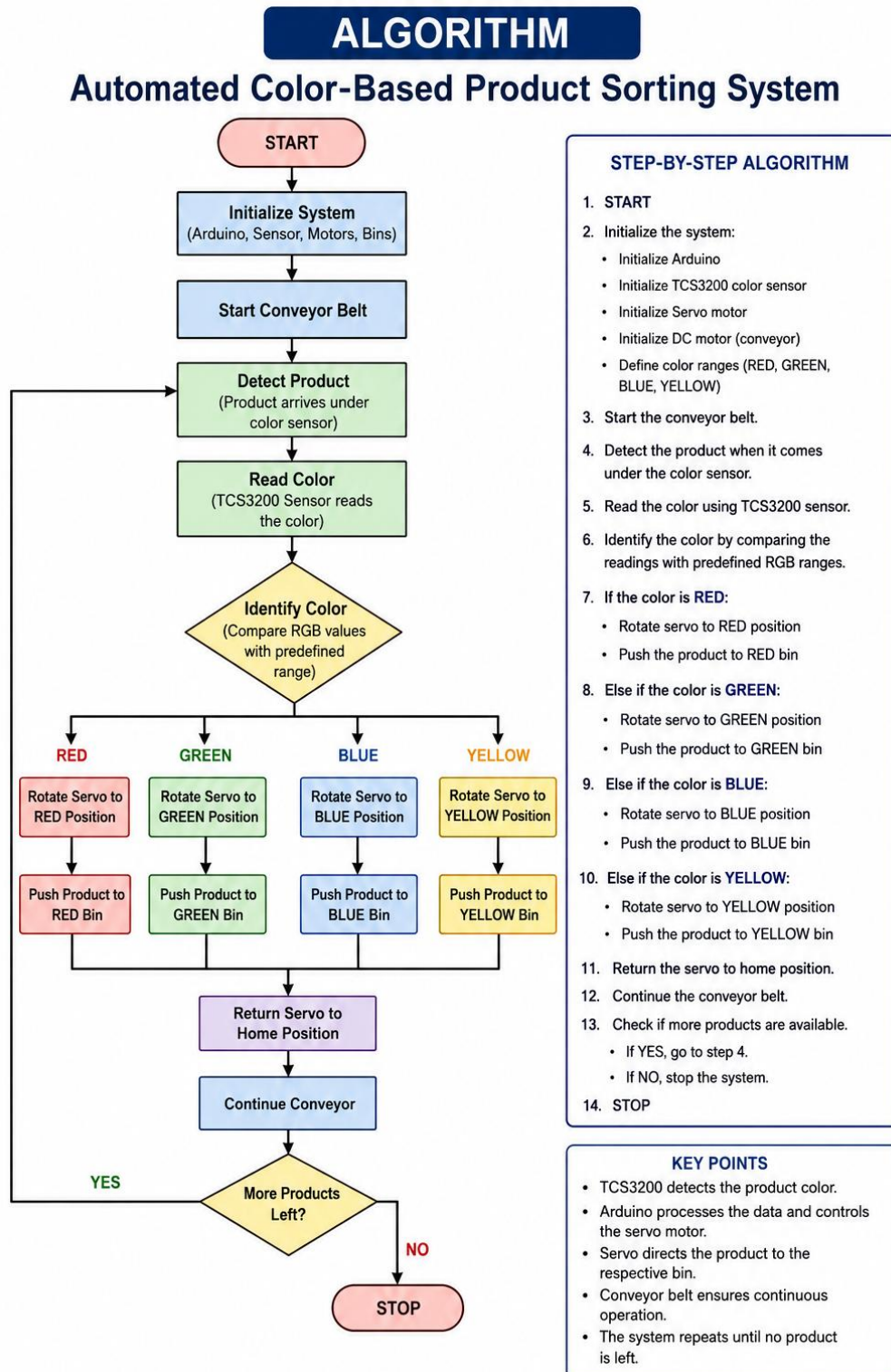
S.No.	Component	Pin/Terminal	Connection	Purpose
1	Arduino UNO	5V, GND	Power supply	Provides power and control
2	TCS3200 Color Sensor	VCC, GND	5V, GND	Sensor power
3	TCS3200	S0, S1	D2, D3	Frequency scaling
4	TCS3200	S2, S3	D4, D5	Color filter selection
5	TCS3200	OUT	D6	Sends detected color signal
6	Servo Motor	Signal	D9	Controls product sorting arm
7	Servo Motor	VCC, GND	5V, GND	Motor power

S.No.	Component	Pin/Terminal	Connection	Purpose
8	DC Motor	+, -	Motor driver	Drives conveyor belt
9	Motor Driver (L298N)	IN1, IN2	D7, D8	Controls conveyor motor direction
10	L298N	OUT1, OUT2	DC Motor	Motor connection
11	L298N	12V, GND	External supply	Supplies motor power
12	LED/Buzzer	Signal	D10	Indicates sorting status

6. Circuit Design



7. Algorithm



8. Coding

```
// =====  
  
// AUTOMATED COLOR BASED PRODUCT SORTING SYSTEM  
  
// ESP8266 + TCS3200 + IR SENSOR + SERVO + LEDs + ThingSpeak  
  
// =====  
  
#include <ESP8266WiFi.h>  
#include <Servo.h>  
  
  
// =====  
  
// WIFI  
  
// =====  
  
const char* ssid = "HITMAN";  
const char* password = "123456789";  
  
  
// =====  
  
// THINGSPEAK  
  
// =====  
  
const char* server = "api.thingspeak.com";  
  
String apiKey = "NNAB5JVGXH3ZZJPN";  
  
  
// =====  
  
// TCS3200  
  
// =====  
  
#define S0 18
```

```
#define S1 19
```

```
#define S2 21
```

```
#define S3 22
```

```
#define OUT_PIN 23
```

```
// =====
```

```
// IR PROXIMITY SENSOR
```

```
// =====
```

```
#define IR_PIN 25
```

```
// =====
```

```
// SERVO
```

```
// =====
```

```
#define SERVO_PIN 26
```

```
Servo sortingServo;
```

```
// =====
```

```
// LEDs
```

```
// =====
```

```
#define RED_LED 27
```

```
#define BLUE_LED 14
```

```
// =====
```

```
// VARIABLES
```

```
// =====
```

```
unsigned long redValue;  
unsigned long greenValue;  
unsigned long blueValue;
```

```
int productCount = 0;
```

```
String color = "UNKNOWN";
```

```
// =====  
// SETUP  
// =====
```

```
void setup()
```

```
{
```

```
  Serial.begin(115200);
```

```
  Serial.println();
```

```
  Serial.println("=====");
```

```
  Serial.println("COLOR SORTING SYSTEM");
```

```
  Serial.println("ESP8266 STARTING...");
```

```
  Serial.println("=====");
```

```
  // -----
```

```
  // TCS3200
```

```
  // -----
```

```
  pinMode(S0, OUTPUT);
```

```
  pinMode(S1, OUTPUT);
```



```
pinMode(S2, OUTPUT);
```

```
pinMode(S3, OUTPUT);
```

```
pinMode(OUT_PIN, INPUT);
```

```
// 20% frequency scaling
```

```
digitalWrite(S0, HIGH);
```

```
digitalWrite(S1, LOW);
```

```
// -----
```

```
// IR SENSOR
```

```
// -----
```

```
pinMode(IR_PIN, INPUT);
```

```
// -----
```

```
// LEDs
```

```
// -----
```

```
pinMode(RED_LED, OUTPUT);
```

```
pinMode(BLUE_LED, OUTPUT);
```

```
digitalWrite(RED_LED, LOW);
```

```
digitalWrite(BLUE_LED, LOW);
```

```
// -----
```

```
// SERVO
```

```
// -----
```

```
sortingServo.attach(SERVO_PIN);

sortingServo.write(90);

// -----
// WIFI
// -----

Serial.println();
Serial.print("Connecting to WiFi");

WiFi.begin(ssid, password);

int attempts = 0;

while (WiFi.status() != WL_CONNECTED && attempts < 30)
{
    delay(500);
    Serial.print(".");
    attempts++;
}

Serial.println();

if (WiFi.status() == WL_CONNECTED)
{
    Serial.println("WiFi Connected!");
    Serial.print("IP Address: ");
```

```
    Serial.println(WiFi.localIP());
}
else
{
    Serial.println("WiFi Connection Failed!");
}

Serial.println();
Serial.println("System Ready");
Serial.println("Place a product near the IR sensor.");
Serial.println();
}

// =====
// MAIN LOOP
// =====

void loop()
{
    // IR sensor normally gives LOW when object is detected
    int objectState = digitalRead(IR_PIN);

    if (objectState == LOW)
    {
        Serial.println();
        Serial.println("*****");
        Serial.println("PRODUCT DETECTED");
        Serial.println("*****");
    }
}
```

```
delay(300);

// Read color
readColor();

// Increase product count
productCount++;

Serial.print("Detected Color: ");
Serial.println(color);

Serial.print("Product Count: ");
Serial.println(productCount);

// Sort product
sortProduct();

// Send data to ThingSpeak
sendThingSpeak();

Serial.println("*****");
Serial.println();

// Wait until product is removed
while (digitalRead(IR_PIN) == LOW)
{
    delay(100);
}
```

```
    delay(1000);  
}
```

```
    delay(100);  
}
```

```
// =====  
// READ COLOR  
// =====
```

```
void readColor()  
{  
    Serial.println("Reading color...");
```

```
    // -----  
    // RED  
    // -----
```

```
    digitalWrite(S2, LOW);  
    digitalWrite(S3, LOW);
```

```
    delay(100);
```

```
    redValue = pulseIn(OUT_PIN, LOW, 100000);
```

```
    // -----  
    // GREEN  
    // -----
```

```
digitalWrite(S2, HIGH);
digitalWrite(S3, HIGH);

delay(100);

greenValue = pulseIn(OUT_PIN, LOW, 100000);

// -----
// BLUE
// -----

digitalWrite(S2, LOW);
digitalWrite(S3, HIGH);

delay(100);

blueValue = pulseIn(OUT_PIN, LOW, 100000);

// Prevent zero values
if (redValue == 0)
    redValue = 9999;

if (greenValue == 0)
    greenValue = 9999;

if (blueValue == 0)
    blueValue = 9999;

// -----
```

```
// DISPLAY SENSOR VALUES

// -----

Serial.println();
Serial.println("TCS3200 VALUES");

Serial.print("Red  = ");
Serial.println(redValue);

Serial.print("Green = ");
Serial.println(greenValue);

Serial.print("Blue  = ");
Serial.println(blueValue);

// -----

// COLOR DECISION

// -----

if (redValue < greenValue && redValue < blueValue)
{
    color = "RED";
}
else if (greenValue < redValue && greenValue < blueValue)
{
    color = "GREEN";
}
else if (blueValue < redValue && blueValue < greenValue)
{

```

```
        color = "BLUE";
    }
    else
    {
        color = "UNKNOWN";
    }
}

// =====

// SORTING

// =====

void sortProduct()
{
    // Turn OFF LEDs
    digitalWrite(RED_LED, LOW);
    digitalWrite(BLUE_LED, LOW);

    // -----

    // RED

    // -----

    if (color == "RED")
    {
        Serial.println("RED PRODUCT");

        digitalWrite(RED_LED, HIGH);

        // Red sorting position
```



```
sortingServo.write(30);
```

```
delay(1000);
```

```
// Return servo
```

```
sortingServo.write(90);
```

```
delay(500);
```

```
digitalWrite(RED_LED, LOW);
```

```
}
```

```
// -----
```

```
// BLUE
```

```
// -----
```

```
else if (color == "BLUE")
```

```
{
```

```
Serial.println("BLUE PRODUCT");
```

```
digitalWrite(BLUE_LED, HIGH);
```

```
// Blue sorting position
```

```
sortingServo.write(150);
```

```
delay(1000);
```

```
// Return servo
```

```
sortingServo.write(90);
```

```
    delay(500);

    digitalWrite(BLUE_LED, LOW);
}

// -----
// GREEN
// -----

else if (color == "GREEN")
{
    Serial.println("GREEN PRODUCT");

    // No green LED available
    // Servo stays at center
    sortingServo.write(90);

    delay(500);
}

// -----
// UNKNOWN
// -----

else
{
    Serial.println("UNKNOWN COLOR");
```

```
    sortingServo.write(90);

    delay(500);
}
}

// =====

// SEND DATA TO THINGSPEAK

// =====

void sendThingSpeak()
{
    if (WiFi.status() != WL_CONNECTED)
    {
        Serial.println("WiFi not connected.");
        return;
    }

    // -----

    // COLOR CODE

    // -----

    int colorCode = 0;

    if (color == "RED")
    {
        colorCode = 1;
    }

    else if (color == "GREEN")
```

```
{
    colorCode = 2;
}
else if (color == "BLUE")
{
    colorCode = 3;
}
else
{
    colorCode = 0;
}

// -----
// SERVO ANGLE
// -----

int servoAngle = 90;

if (color == "RED")
{
    servoAngle = 30;
}
else if (color == "BLUE")
{
    servoAngle = 150;
}

// -----
// CREATE URL
```

```
// -----
```

```
String url = "/update?api_key=";
```

```
url += apiKey;
```

```
url += "&field1=";
```

```
url += String(colorCode);
```

```
url += "&field2=";
```

```
url += String(redValue);
```

```
url += "&field3=";
```

```
url += String(greenValue);
```

```
url += "&field4=";
```

```
url += String(blueValue);
```

```
url += "&field5=";
```

```
url += String(productCount);
```

```
url += "&field6=";
```

```
url += String(servoAngle);
```

```
// -----
```

```
// CONNECT TO THINGSPEAK
```

```
// -----
```

```
WiFiClient client;
```

```
Serial.println("Sending data to ThingSpeak...");

if (!client.connect(server, 80))
{
    Serial.println("ThingSpeak connection failed.");
    return;
}

client.print(String("GET ") + url + " HTTP/1.1\r\n" +
    "Host: " + server + "\r\n" +
    "Connection: close\r\n\r\n");

delay(100);

// -----
// READ RESPONSE
// -----

while (client.available())
{
    String line = client.readStringUntil('\r');

    Serial.print(line);
}

client.stop();

Serial.println();
Serial.println("ThingSpeak update completed.");
```

}

9. System Working

The Automated Color-Based Product Sorting System works by continuously detecting and sorting products according to their color.

1. **Product Feeding:** Products are placed on the conveyor belt.
2. **Conveyor Movement:** A DC motor drives the conveyor belt and moves the products toward the sensing area.
3. **Color Detection:** The TCS3200 color sensor detects the color of each product.
4. **Data Processing:** The Arduino UNO receives the sensor output and compares it with predefined color values.
5. **Color Identification:** The system identifies the product as Red, Green, Blue, or Yellow.
6. **Sorting Action:** The Arduino sends a control signal to the servo motor.
7. **Product Separation:** The servo mechanism pushes or directs the product into its corresponding color bin.
8. **Servo Reset:** After sorting, the servo returns to its original position.
9. **Continuous Operation:** The conveyor continues moving and the process repeats for the next product.

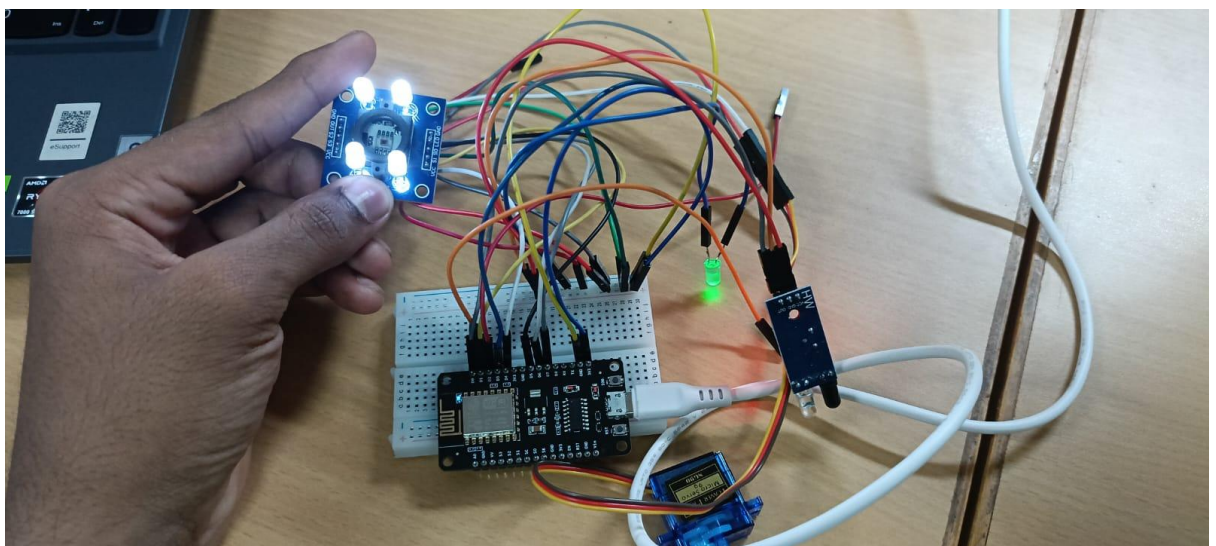
10. Bill of Materials

S.No.	Component	Quantity	Purpose
1	Arduino UNO	1	Main controller
2	TCS3200 Color Sensor	1	Detects product color
3	DC Gear Motor	1	Drives conveyor belt
4	L298N Motor Driver	1	Controls DC motor
5	Servo Motor (SG90/MG996R)	1	Sorts products into bins
6	Conveyor Belt	1	Transports products
7	DC Power Supply	1	Provides electrical power
8	Sorting Bins	4	Collects different colored products
9	Jumper Wires	1 Set	Electrical connections
10	Breadboard/PCB	1	Circuit assembly
11	Resistors	As required	Signal/indicator connections
12	LED Indicators	4	Indicates detected color
13	Buzzer	1	Provides status indication
14	Connecting Cables	As required	Component connections
15	Mechanical Frame	1	Supports conveyor and components

11. Prototype Implementation

The prototype was developed as a small-scale automated conveyor-based sorting system to demonstrate color detection and automatic product separation.

- A mechanical frame and conveyor belt were assembled to transport the products.
- A DC gear motor with L298N motor driver was connected to the Arduino UNO to control conveyor movement.
- The TCS3200 color sensor was mounted above the conveyor to detect the color of passing products.
- The Arduino UNO was programmed to process the sensor readings and identify different colors such as Red, Green, Blue, and Yellow.
- A servo motor with a sorting arm was installed near the end of the conveyor.
- Separate collection bins were positioned for each color.
- When a product reaches the sensor, its color is detected and processed by the Arduino.
- The servo motor then moves to the required position and directs the product into the corresponding color bin.
- After sorting, the servo returns to its initial position and the conveyor continues moving.
- The prototype was tested using multiple colored products to verify color detection, sorting accuracy, and continuous operation.



12. Results & Performance Analysis

Results

The developed Automated Color-Based Product Sorting System successfully detected and sorted products according to their colors. The TCS3200 color sensor identified the product color, while the Arduino UNO processed the sensor data and controlled the servo motor for automatic sorting.

The prototype successfully demonstrated the separation of Red, Green, Blue, and Yellow products into their respective bins. The system reduced manual effort and provided consistent sorting during continuous operation.

Performance Analysis

Parameter	Observed Performance
Color Detection	Successfully detects different colors
Sorting Accuracy	High under controlled lighting conditions
Sorting Speed	Suitable for small-scale automation
Human Effort	Significantly reduced
Sorting Method	Automatic
Operation	Continuous
Reliability	Good for prototype applications
Industrial Application	Suitable for basic manufacturing and packaging

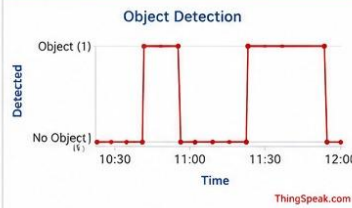
Overall Performance

The prototype demonstrates that color-based automatic sorting can improve productivity, reduce human errors, and provide consistent product separation. For industrial implementation, performance can be further improved by using better industrial-grade color sensors, higher-speed conveyors, pneumatic actuators, and controlled lighting conditions.

Field 1 Chart



Field 2 Chart



Field 3 Chart



Field 4 Chart



Field 5 Chart



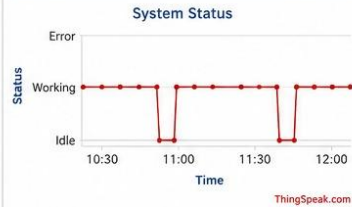
Field 6 Chart



Field 7 Chart



Field 8 Chart



5. Results

- ✓ Live monitoring on ThingSpeak
- ✓ Real-time data update
- ✓ Graphs for all parameters
- ✓ Easy to analyze performance